

Day 1

Spring Boot From Zero

A beginner-friendly study note for understanding your first Java backend without getting buried in terminology.



DAY 1 TARGET

By the end of today, you should be able to build a small Spring Boot API, explain the request flow in plain English, and recreate the project without copying the solution.

LEARN Understand the idea	BUILD Write the code	REBUILD Do it from memory
-------------------------------------	--------------------------------	-------------------------------------

01 What are we actually building?

Forget Spring for a moment. A backend is a program that waits for requests, does some work, and sends a response.



Example: a browser or Postman asks for GET /api/hello. Your Java application finds the code responsible for that path, runs it, and sends something back.

Plain-English definition

Backend = code that receives requests, performs application work, and returns useful responses.

02 What is Spring Boot?

Spring Boot is a convenient way to build Java backend applications. You still write Java. Spring Boot handles a lot of the setup that you would otherwise have to do yourself.

JAVA The language you write code in.	SPRING A collection of tools for Java applications.	SPRING BOOT An easier way to set up and run Spring applications.
--	---	--

Remember

Spring Boot does not replace Java. You write Java code; Spring Boot helps turn that code into a working backend application.

03 Dependency Injection — the simple idea

The name sounds difficult, but the idea is straightforward. Suppose a Car needs an Engine.

Option A — Car creates the Engine itself:

```
class Car {  
    private Engine engine = new Engine();  
}
```

Option B — the Engine is given to the Car:

```
class Car {  
    private Engine engine;  
  
    Car(Engine engine) {  
        this.engine = engine;  
    }  
}
```

The idea to remember

Dependency Injection means a class receives the objects it needs instead of creating those objects itself.

Spring can create those objects and give them to the classes that need them.

04 Bean and ApplicationContext

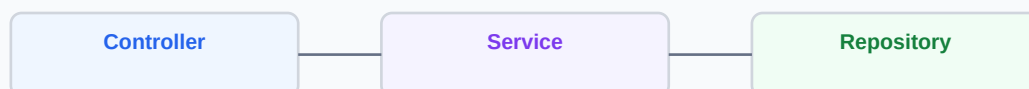
A **Bean** is simply an object that Spring creates and manages.

```
@Service  
public class UserService {  
}
```

Spring can create a UserService object and keep track of it.

SPRING APPLICATION CONTEXT

A central place where Spring keeps track of objects it manages.



ApplicationContext

Think of it as Spring's central place for keeping track of the objects it manages.

05 The first Spring annotations

Annotation	Plain-English meaning
<code>@RestController</code>	This class handles web requests.
<code>@Service</code>	This class contains application/business work.
<code>@GetMapping</code>	Run this method when a GET request reaches this path.
<code>@SpringBootApplication</code>	Marks the main class used to start the Spring Boot application.

06 Create the project

Use Spring Initializr and choose:

Project	Maven
Language	Java
Packaging	Jar
Dependencies	Spring Web + Spring Boot Actuator

Open the generated project in IntelliJ IDEA, Eclipse, or VS Code.

07 Understand the project structure

<code>src/main/java</code>	Your Java source code.
<code>src/main/resources</code>	Configuration and other resources.
<code>src/test</code>	Automated tests.
<code>pom.xml</code>	Maven's project file: libraries, plugins, and build information.

Maven is the helper that downloads libraries and helps build, test, and package the project.

08 The main Spring Boot class

```
@SpringBootApplication
public class TaskApiApplication {

    public static void main(String[] args) {
        SpringApplication.run(
            TaskApiApplication.class,
            args
        );
    }
}
```

For today, read this as:

“Start my Spring Boot application.”

After startup, your application normally listens on `localhost:8080`.

09 Build your first endpoint

```
@RestController
public class HelloController {

    @GetMapping("/hello")
    public String hello() {
        return "Hello Tanmoy!";
    }
}
```



`@GetMapping("/hello")` means: when a GET request reaches `/hello`, run this method.

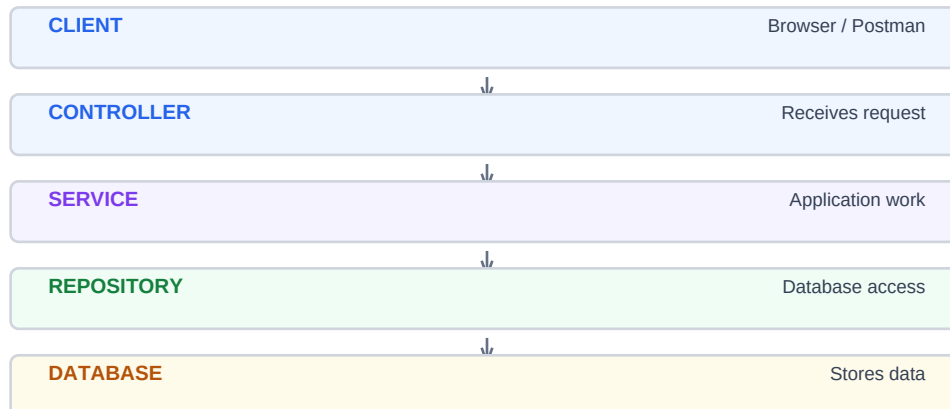
10 HTTP methods — only the basics today

GET	Give me data
POST	Create data
PUT	Replace/update data
PATCH	Partially update data
DELETE	Delete data

Focus on GET today. The others will become important as you build CRUD APIs.

11 Why Controller → Service → Repository?

A tiny project can put everything in one class. A real project grows. Separating responsibilities makes each part easier to understand and change.



Controller	Receives the web request.
Service	Contains the application work.
Repository	Later, handles database access.

12 Move the work into a Service

```
@Service
public class HelloService {

    public String getMessage() {
        return "Hello Tanmoy!";
    }
}

@RestController
public class HelloController {

    private final HelloService helloService;

    public HelloController(HelloService helloService) {
        this.helloService = helloService;
    }

    @GetMapping("/hello")
    public String hello() {
        return helloService.getMessage();
    }
}
```

Important

Notice that the Controller does not use `new HelloService()`. Spring supplies the Service object. This pattern is called constructor injection.

13 DTO and JSON

JSON is a common format for sending data between applications:

```
{  
  "message": "Hello Tanmoy!"  
}
```

A **DTO** is simply a Java object whose job is to carry data.

```
public class HelloResponse {  
  
    private String message;  
  
    public HelloResponse(String message) {  
        this.message = message;  
    }  
  
    public String getMessage() {  
        return message;  
    }  
}
```

Remember

DTO = an object used to carry data.

14 application.yml — application settings

This file normally lives at:

```
src/main/resources/application.yml
```

```
spring:
  application:
    name: task-api

server:
  port: 8080
```

Think of this as a settings file. Keeping settings outside your Java code makes it easier to run the same application in different environments.

15 Development vs Production

DEV

application-dev.yml

Your laptop / development

PROD

application-prod.yml

Real deployed application

A **profile** lets Spring Boot use different settings for different environments. You only need the concept today.

16 Actuator — is my application alive?

Spring Boot Actuator provides useful management endpoints. The one to learn today is:

```
GET /actuator/health
```

A healthy application may return:

```
{
  "status": "UP"
}
```

Plain-English meaning

“The application is running and responding.”

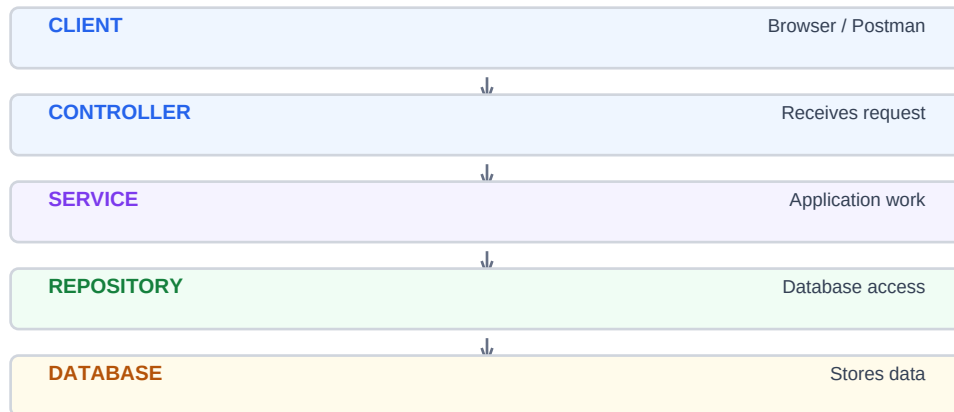
17 Logging

INFO	Normal information
WARN	Something deserves attention
ERROR	Something went wrong
DEBUG	Extra detail while investigating

Logs help you see what the application is doing and diagnose problems.

18 Your Day 1 project — Task API

Endpoint	Purpose
GET /api/hello	Return a JSON greeting.
GET /actuator/health	Check application health.



For today's implementation, keep it small: **Controller** → **Service** → **DTO**. The Repository and Database come later.

19 Run and test

```
# Windows
mvnw.cmd spring-boot:run

# macOS / Linux
./mvnw spring-boot:run
```

Then test these in your browser or Postman:

```
http://localhost:8080/api/hello
http://localhost:8080/actuator/health
```

20 Git and README

```
git init
git status
git add .
git commit -m "Create initial Spring Boot API"
```

Your README should explain what the project is, what technologies it uses, the endpoints, and how to run it.

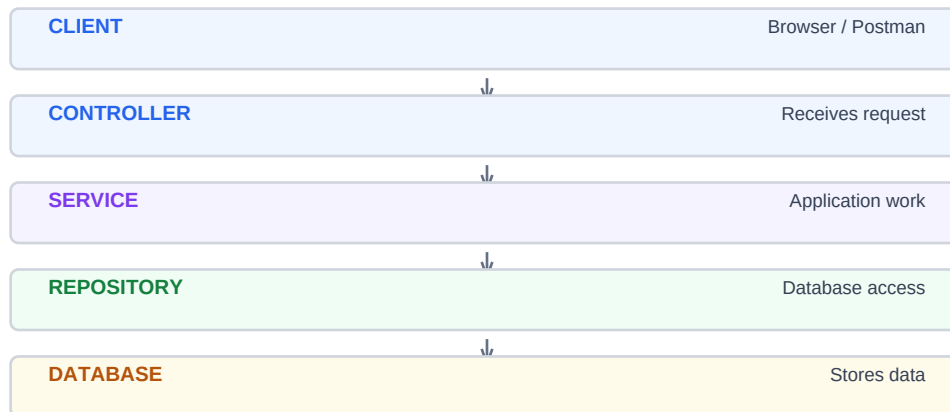
21 Practice exercises

1. Create GET /api/greeting using GreetingController, GreetingService, and GreetingResponse.
2. Create GET /api/student returning name and course using Controller → Service → DTO.
3. Inject MessageService into MessageController. Do not create it with new MessageService().

4. Add `app.name` and `app.version` to configuration and experiment with reading them.

5. **Challenge:** Close this PDF and recreate `GET /api/hello` from memory.

22 The mental model to remember



One sentence

Request comes in → Controller receives it → Service does the work → data comes back → response goes out.

When we add a database, the Repository sits between the Service and the Database.

23 Day 1 checklist

■ I know what Spring Boot is.	■ I understand Dependency Injection in simple terms.
■ I know what a Bean is.	■ I roughly understand ApplicationContext.
■ I know what Maven does.	■ I created and ran a Spring Boot project.
■ I created a Controller and a Service.	■ I used constructor injection.
■ I understand DTO and JSON.	■ I created /api/hello.
■ I tested /actuator/health.	■ I understand application.yml.
■ I understand why profiles exist.	■ I initialized Git and wrote a README.
■ I rebuilt the endpoint without copying it.	

24 Day 1 interview / self-check

25 How to study Spring Boot

1 Learn the idea	2 Understand why	3 Write code	4 Break it	5 Fix it
---------------------	---------------------	-----------------	---------------	-------------

Day 1 success condition

You do not need to memorize every Spring annotation. If you can rebuild the small API tomorrow without copying the code and explain the request flow in your own words, you have successfully completed Day 1.